Ordinal Logistic Regression with Split Conformal Prediction

for Clothing Fit Recommendation
A classical machine learning pipeline with calibrated uncertainty

Patrick Oliveira

October 15, 2026

Abstract

We predict clothing fit (*small*, *fit*, *large*) as an *ordinal* classification problem using a proportional-odds cumulative link model, trained by maximizing the likelihood with `scipy.optimize` – classical machine learning, with no neural network anywhere in the pipeline. We wrap the fitted model in split conformal prediction, implemented from scratch in NumPy, to turn a bare point prediction into a calibrated *set* of plausible fit outcomes with a distribution-free marginal coverage guarantee. Users are split at the group level (train/validation/test share no user) so the model is evaluated on genuinely unseen shoppers, not on memorized body measurements. On a synthetic fixture matching the real dataset's clustering and class skew, the ordinal model attains a mean absolute ordinal error of 0.3535 versus 0.4397 for a majority-class floor and 0.6298 for an unordered multinomial logistic regression baseline with identical features, and the conformal wrapper achieves 88.55% empirical coverage against a 90% target. We state explicitly why standard ranking metrics (NDCG, Recall@K, MAP) do not apply to this problem.

Contents

1 Notation **3**

2 Introduction **3**
2.1 Two contributions, stated precisely . . . 3
2.2 Evaluation honesty . . . 3
2.3 Repository . . . 3

3 Data **4**
3.1 Source . . . 4
3.2 Synthetic fixture . . . 4
3.3 Grouped user split . . . 4

4 Feature engineering **4**
4.1 Static versus historical aggregate: the distinction that matters . . . 4
4.2 Point-in-time construction . . . 5
4.3 Leakage tests . . . 6
4.4 Context features . . . 6
4.5 Training/serving parity . . . 6

5 Methodology **6**
5.1 Cumulative link (proportional-odds) model . . . 6
5.2 Negative log-likelihood objective . . . 7
5.3 Baselines . . . 7
5.4 Split conformal prediction . . . 7
5.5 Why this problem is calibration, not ranking . . . 8
5.6 Serving . . . 8

6 Evaluation protocol **8**
6.1 Partitioning . . . 8
6.2 Metrics . . . 8
6.3 Model selection and test discipline . . . 9
6.4 Training/serving parity . . . 9

7 Results **9**
7.1 Reading the headline number . . . 10

8 Conclusion **10**
8.1 Limitations . . . 10
8.2 What a reader should take away . . . 11

1 Notation

Let K denote the number of ordered fit levels; throughout, $K = 3$ with levels $\{\text{small}, \text{fit}, \text{large}\}$ encoded as $y \in \{0, 1, 2\}$ in that order. A feature vector $\mathbf{x} \in \mathbb{R}^d$ describes one (user, item, requested size) review. The logistic function is $\sigma(z) = (1 + e^{-z})^{-1}$. Weights are written $\mathbf{w} \in \mathbb{R}^d$ and ordered thresholds $\theta_1 < \theta_2 < \cdots < \theta_{K-1}$.

2 Introduction

Most tabular recommender portfolios that touch an explicit, multi-level outcome treat it as an unordered multinomial classification problem, or collapse it into a binary "good/bad" signal and reach for a ranking metric. Clothing-fit prediction gives a clean counter-example to that default. A garment reported as *small*, *fit*, or *large* is not three unrelated categories: the levels are strictly ordered, and the cost of confusing *small* for *large* is qualitatively worse than confusing *small* for *fit*. A model that ignores this ordering discards information the problem handed it for free.

2.1 Two contributions, stated precisely

This report accompanies a working repository, not a novel method. Its two contributions are methodological:

1. **Ordinal structure, not multinomial.** We fit a proportional-odds cumulative link model (Section 5) instead of treating the three fit levels as unordered classes, and we show this pays off on ordinal-aware metrics (Section 7) even though both models see identical features.

2. **Calibrated uncertainty, not a bare label.** Wrapping the fitted model in split conformal prediction turns a point guess into a *set* of plausible outcomes with a finite-sample marginal coverage guarantee (Section 5). For a sizing recommendation, "this could run fit or large" is a more honest and more actionable signal than a single confident label that is sometimes simply wrong.

Both pieces are classical machine learning: the cumulative link model is fit by maximizing a smooth, explicit likelihood with `scipy.optimize.minimize`, and the conformal calibration is closed-form arithmetic over a held-out calibration set. No neural network, embedding learned by gradient descent, or deep framework appears anywhere in the pipeline.

2.2 Evaluation honesty

Users are split at the group level: every review by a given user lands in exactly one of train, validation, or test (Section 3). This prevents the model from memorizing one user's exact body-measurement combination and matching it back to that user's historical fit outcomes – a leakage mode specific to supervised tabular problems with repeated subjects, distinct from the temporal point-in-time leakage that recommender systems usually worry about. Both leakage modes are present in this dataset and are tested independently (Section 4).

Hyperparameters for every model, baselines included, are selected on the validation partition; the test partition is read once, to produce Table 2.

2.3 Repository

The implementation and the scripts that generate every table in this document are available in the accompanying repository. The results table (Table 2) is emitted by `scripts/export_results_table.py` and the feature catalogue (Table 1) by `scripts/export_feature_tables.py`, so the document cannot drift away from the code.

3 Data

3.1 Source

The target dataset is the Clothing Fit Dataset for Size Recommendation (Kaggle: `rmisra/clothing-fit-datas` covering garment reviews from online rental and retail marketplaces. Each review carries static shopper body measurements (height, weight, bust size, body type, age), an item identifier and category, the purchased garment size, and the ordinal outcome `fit` $\in$ {small, fit, large}.

3.2 Synthetic fixture

`scripts/download_data.py` defaults to a synthetic generator (no Kaggle account required, so the full pipeline – including CI – runs from a clean checkout) that reproduces the dataset's structural properties rather than its exact values:

- **Clustered users and items.** Each user has a latent "true size"; each item has a nominal size and a fit sensitivity. Users self-select garments close to their own size (as real shoppers do), with enough spread that mismatches still occur regularly.
- **Skewed fit distribution.** The probability of *small* or *large* rises smoothly with the gap between a user's true size and the purchased item's nominal size, scaled by the item's sensitivity. The resulting label distribution is skewed toward *fit* (56.0% fit, 23.3% small, 20.7% large in the fixture used for Table 2), matching the real dataset's class imbalance.
- **True chronology.** Every review carries a `review_date`, so the point-in-time machinery (Section 4) is exercised against real ordering, not an append-order proxy.

A `-kaggle` flag downloads the real dataset via the Kaggle API when `KAGGLE_USERNAME`/`KAGGLE_KEY` are set in the environment; credentials are never read from or written to the repository.

3.3 Grouped user split

Train, validation and test partitions are formed by randomly assigning *users* (not rows) to a partition, so every review by a given user lands in exactly one partition (`recommender.data.splitting.grouped_u` This is distinct from – and in addition to – the point-in-time cut-off described in Section 4: it specifically prevents a model from learning "this exact combination of height/weight/body-type always runs small" from a user's train-partition reviews and then trivially matching it against the same user's test-partition reviews. `GroupSplit` raises if construction ever produces an overlapping user across partitions, and this is asserted directly in `tests/test_features_leakage.py`.

4 Feature engineering

Table 1 lists every declared feature, generated directly from `conf/features.yaml` – the single source of truth consumed by both the training job and the serving path (Section 5.6).

4.1 Static versus historical aggregate: the distinction that matters

This dataset makes a distinction that is easy to blur and important to get right.

Static attributes – height, weight, BMI, bust band and cup, body type, age, item category – are known the moment a user fills in a profile or an item enters the catalogue. They carry *no* point-in-time cut-off because there is no "future" version of a person's height to leak: these values simply exist before any review is written, train or test alike. `recommender.features.user_features` and `item_features.item_category` read them directly off the row being featurized.

Table 1: Engineered features. Static attributes need no cut-off; historical aggregates are computed strictly before the row being predicted (point-in-time).

Feature	Family	Type	Point-in-time
`user_height_cm`	user	static	–
`user_weight_kg`	user	static	–
`user_bmi`	user	static	–
`user_bust_band`	user	static	–
`user_bust_cup_ordinal`	user	static	–
`user_body_type`	user	static	–
`user_age`	user	static	–
`item_category`	item	static	–
`item_nominal_size`	item	static	–
`user_prior_review_count`	interaction	historical aggregate	✓
`user_prior_mismatch_rate`	interaction	historical aggregate	✓
`user_prior_small_rate`	interaction	historical aggregate	✓
`item_prior_mismatch_rate`	interaction	historical aggregate	✓
`context_rented_for`	context	static	–

Historical aggregates – a user's or an item's past mismatch rate, and a user's prior review count – are a fundamentally different kind of feature: they summarize *outcomes of other reviews*, and the review being predicted is one such outcome. If a user's "prior mismatch rate" were computed over *all* of that user's reviews (including the one being predicted, or ones that happened later), the feature would partially encode the label it is meant to help predict. This is exactly the leak this project is built to avoid.

4.2 Point-in-time construction

Each review carries two chronological positions, assigned once when the dataset is indexed (`recommender.data.loader.build_indexed_dataset`, `recommender.features.pipeline.assign_global_o`

- `review_order` – the review's position within its own user's history (0, 1, 2, ...), used by the two user-level aggregates.
- `global_order` – the review's position within the entire dataset's chronology (by `review_date`), used by the item-level aggregate, since an item can be reviewed by many different users across every partition.

For a review at position k (per the relevant order column), the historical aggregate is an *expanding* statistic over strictly prior positions only:

$$\widehat{r}(k) = \frac{1}{k} \sum_{j<k} \mathbb{1}[\text{outcome}_j \text{ was a mismatch}], \qquad \widehat{r}(0) \text{ undefined (no prior history).} \tag{1}$$

The strict inequality is the entire contract. A review at a given position can see every review strictly before it, never the review at its own position, and never one after it. This is enforced by computing the statistic over a frame sorted by the order column and using an explicit less-than comparison, not a window function that could silently include the current row (`interaction_features._expanding_prior_mean`).

Rows with no qualifying prior history (a user's or item's very first appearance) get `NaN` from the raw computation; the feature pipeline then applies the fill policy declared in `conf/features.yaml` – the global mismatch rate for the two rate features, zero for the count and the small-rate feature, since a user's first review has, by definition, no observed directional bias yet.

4.3 Leakage tests

`tests/test_features_leakage.py` operationalizes the point-in-time contract directly: it injects a fabricated future review (an extreme, label- flipping outcome) for an existing user or item, strictly after the real cut-off, and asserts that every feature value for rows at or before that cut-off is unchanged. A second test asserts the point-in-time aggregate differs from a naive whole-history average whenever a user has more than one review – if they ever coincided, every row would be seeing its own future.

4.4 Context features

A shopper-stated occasion (`context_rented_for`, e.g. "wedding", "work") is supplied at request time, like a search filter. It is information about the current transaction, not a historical aggregate, so it needs no cut-off either – it exists in exactly the same sense static attributes do.

4.5 Training/serving parity

`FeatureStore.transform` (batch) and `FeatureStore.row_vector` (single serving-time row) are the only two entry points into the feature pipeline, and `tests/test_features_parity.py` asserts they produce numerically identical output for the same logical row – the failure mode this guards against is an offline batch computing one feature vector while an online request computes a different one for what should be the same input.

5 Methodology

5.1 Cumulative link (proportional-odds) model

For an ordinal outcome with K levels and feature vector $\mathbf{x}$, the cumulative-link model [4, 1] with a logistic link assumes

$$P(y \le k \mid \mathbf{x}) = \sigma(\theta_k - \mathbf{w}^\top \mathbf{x}), \qquad k = 1, \dots, K-1, \tag{2}$$

with ordered thresholds $\theta_1 < \theta_2 < \cdots < \theta_{K-1}$ and a single weight vector $\mathbf{w}$ shared across every threshold – the "proportional odds" / "parallel lines" assumption: a one-unit increase in any feature shifts every cumulative log-odds by the same amount $-\mathbf{w}^\top \mathbf{x}$, which is what makes the model genuinely ordinal rather than $K-1$ unrelated logistic regressions. By convention $P(y \le 0 \mid \mathbf{x}) = 0$ and $P(y \le K-1 \mid \mathbf{x}) = 1$, so the per-class probability is the first difference of the cumulative distribution,

$$P(y = c \mid \mathbf{x}) = P(y \le c \mid \mathbf{x}) - P(y \le c-1 \mid \mathbf{x}), \qquad c = 0, \dots, K-1. \tag{3}$$

Unconstrained parametrization. An optimizer searching over $(\theta_1, \dots, \theta_{K-1})$ directly could produce a non-monotonic, invalid threshold vector. We instead optimize over an unconstrained vector $\boldsymbol{\delta} \in \mathbb{R}^{K-1}$ and set

$$\theta_1 = \delta_1, \qquad \theta_k = \theta_{k-1} + \mathrm{softplus}(\delta_k) \;\; (k = 2, \dots, K-1), \tag{4}$$

with $\mathrm{softplus}(z) = \log(1 + e^z) > 0$. Because every gap after the first is a strictly positive softplus output, $\theta_1 < \theta_2 < \cdots < \theta_{K-1}$ holds for *any* real-valued $\boldsymbol{\delta}$, so an unconstrained quasi-Newton optimizer can never propose an invalid threshold ordering. This is implemented in `recommender.models.ordinal._thresholds_from_raw`.

5.2 Negative log-likelihood objective

Given training data $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, parameters $\mathbf{w}$ and $\boldsymbol{\theta}$ are fit by minimizing the ℓ_2-penalized negative log-likelihood

$$\mathcal{L}(\mathbf{w}, \boldsymbol{\theta}) = -\sum_{i=1}^{n} \log P(y = y_i \mid \mathbf{x}_i; \mathbf{w}, \boldsymbol{\theta}) \; + \; \frac{\lambda}{2}\|\mathbf{w}\|_2^2, \tag{5}$$

with $P(y = y_i \mid \cdot)$ given by Equation 3 substituted into Equation 2. The thresholds are not penalized, since shrinking a cut-point toward zero has no natural interpretation the way shrinking a feature weight does. Equation 5 is minimized directly with `scipy.optimize.minimize` (L-BFGS-B) over the unconstrained parameter vector $(\delta_1, \ldots, \delta_{K-1}, \mathbf{w})$ – classical convex-leaning optimization, with no gradient-descent-trained embedding or neural layer anywhere in the model (`recommender.models.ordinal.OrdinalLogisticRegression`).

5.3 Baselines

Majority class. Predicts the single most frequent training label for every row, ignoring features entirely. The floor any model using features must clear.

Multinomial logistic regression. The same engineered features, fed into an *unordered* softmax classifier,

$$P(y = c \mid \mathbf{x}) = \frac{\exp(\mathbf{w}_c^\top \mathbf{x})}{\sum_{c'=0}^{K-1} \exp(\mathbf{w}_{c'}^\top \mathbf{x})}, \tag{6}$$

fit by the same ℓ_2-penalized NLL minimization via `scipy.optimize`. Because it assigns an independent weight vector $\mathbf{w}_c$ to every class, it cannot exploit the ordering the way the proportional-odds model's shared $\mathbf{w}$ does – isolating the ordinal structure's effect, rather than any difference in optimizer or feature set, as the variable under test (Section 7).

5.4 Split conformal prediction

Any fitted classifier that outputs $\hat{P}(y \mid \mathbf{x})$, ordinal or not, can be wrapped in split conformal prediction to produce a prediction *set* with a finite-sample marginal coverage guarantee – implemented from scratch in NumPy in `recommender.models.conformal`.

Nonconformity score. On a calibration set $\mathcal{D}_{\text{cal}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$, disjoint from both the training and test partitions (here: the validation partition), the nonconformity score for example i is

$$\alpha_i = 1 - \hat{P}(y_i \mid \mathbf{x}_i), \tag{7}$$

large exactly when the fitted model placed little probability mass on the true label.

Calibration procedure. For a target miscoverage rate $\alpha \in (0, 1)$, let

$$\hat{q} = \text{the } \lceil (m+1)(1-\alpha) \rceil / m\text{-th empirical quantile of } \{\alpha_1, \ldots, \alpha_m\}. \tag{8}$$

For a new point $\mathbf{x}$, the conformal prediction set is

$$C(\mathbf{x}) = \left\{ y \in \{0, \ldots, K-1\} : 1 - \hat{P}(y \mid \mathbf{x}) \le \hat{q} \right\}. \tag{9}$$

Coverage guarantee. Under the sole assumption that the calibration example and the test example are exchangeable (a strictly weaker assumption than any distributional or parametric one),

$$P(y \in C(\mathbf{x})) \;\geq\; 1 - \alpha, \tag{10}$$

marginally over the randomness in both the calibration set and the new test point [6, 3, 2]. This guarantee is model-agnostic: it holds regardless of whether $\hat{P}$ is well-calibrated, over-confident, or under-confident, which is precisely why conformal prediction is the right tool to pair with a point-prediction model whose own calibration is not otherwise guaranteed.

5.5 Why this problem is calibration, not ranking

NDCG, Recall@K and MAP score a *ranked list of items* against a known set of relevant ones per user, under an implicit assumption that the consumer will see the list top-to-bottom. Nothing in this problem has that shape: every example is one independent (user, item, requested size) review with a single explicit ordinal label, not a list to be cut off at K. There is no notion of "relevant items for this user" to rank against, and no top-K truncation that would make a ranking metric meaningful. We therefore report ordinal-aware classification and calibration metrics (Section 6) instead of adapting a ranking metric that does not fit the problem's shape.

5.6 Serving

The fitted model, its scaler, and its calibrated conformal wrapper are applied identically whether scoring a batch (training/evaluation) or a single incoming request (`recommender.serving.predictions.build_`
Rather than running inference inside the request path, predictions for every anticipated `(user, item, size)` triple are precomputed and stored in DynamoDB keyed by a composite request id; the inference Lambda (`recommender.serving.handler.lambda_handler`) is then a pure key lookup, bounding p99 latency by DynamoDB rather than by feature reconstruction cost.

6 Evaluation protocol

6.1 Partitioning

Users are randomly assigned to train (70%), validation (15%) and test (15%), and every review by a given user lands in exactly one partition (`grouped_user_split`, Section 3). The split is reconstructed and checked for user overlap on every call (`GroupSplit.__post_init__`), and `tests/test_features_leakage.py` asserts that attempting to build a split from row-sliced (rather than user-grouped) partitions raises.

6.2 Metrics

Mean absolute ordinal error. With $y_i \in \{0, 1, 2\}$ the true level and $\hat{y}_i$ the predicted level,

$$\mathrm{MAE}_{\mathrm{ord}} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|. \tag{11}$$

Unlike plain accuracy, this distinguishes a *small*-predicted-as-*large* error (distance 2) from a *small*-predicted-as-*fit* error (distance 1) – the entire reason to treat the problem as ordinal.

Accuracy within one level. The fraction of predictions at most one ordinal step from the truth, $\frac{1}{n} \sum_i \mathbb{1}[|y_i - \hat{y}_i| \leq 1]$ – a tolerant metric that still distinguishes a model that is occasionally off by one level from one that confuses *small* and *large* outright.

Table 2: Point-prediction quality on the held-out test partition. Hyperparameters for every model were selected on the validation partition; the test partition was read once.

Model	Accuracy	MAE (ordinal)	Acc. within 1	Macro F1
OrdinalLR	0.6509	**0.3535**	0.9956	0.5635
Majority	0.5603	0.4397	1.0000	0.2394
MultinomialLR	0.4258	0.6298	0.9444	0.4283

Macro F1. Unweighted mean of per-class F1, reported because accuracy alone is a weak signal under the dataset's class skew (the majority baseline attains high accuracy and within-one-level scores by predicting "fit" for everyone, but a macro F1 near the single-class floor).

Conformal coverage and average set size. Empirical fraction of test examples whose true label is included in the conformal set (Equation 9), and the mean set cardinality. Coverage should sit near the target $1 - \alpha$; set size is the cost of that guarantee – a trivial predictor could "achieve" perfect coverage by returning every class every time (as the majority baseline does in Table 3), which is why set size must always be read alongside coverage, not instead of it.

What we do not report, and why. NDCG@K, Recall@K and MAP are absent from this report by design, not omission; see Section 5.5 for the argument.

6.3 Model selection and test discipline

Every model's hyperparameters (ℓ_2 strength for both logistic models) were selected by inspecting validation-partition metrics; the test partition is touched only once, by `scripts/export_results_table.py`, to produce Table 2. The conformal calibration set is the validation partition, disjoint from both the training data the model's *parameters* were fit on and the test data its *coverage* is measured on – using the same partition for fitting the model and calibrating the conformal wrapper would violate the exchangeability assumption the coverage guarantee (Equation 10) depends on.

6.4 Training/serving parity

`tests/test_serving.py` asserts every precomputed payload's predicted class is a member of its own conformal set (the fitted model's own top choice can never be excluded by its own calibrated threshold), that class probabilities sum to one, that request ids are unique, and that the full payload is JSON-serializable – the same check the reference BPR/SVD portfolio projects use to catch NumPy-scalar serialization failures before they reach a Lambda deployment.

7 Results

Table 2 reports point-prediction quality on the held-out test partition. The ordinal model attains the lowest mean absolute ordinal error by a wide margin over both baselines, confirming that the proportional-odds structure is doing real work: the multinomial baseline sees identical features but, lacking the shared-weight ordinal constraint, distributes its probability mass less efficiently across adjacent levels and pays for it on every ordinal-aware metric. The majority baseline's accuracy-within-one score of 1.0000 is a direct consequence of always predicting "fit" (the dataset's modal class) under a tolerance of one level either side – illustrating exactly why within-one accuracy must be read alongside mean absolute error and macro F1, not instead of them: its macro F1 of 0.2394 is the lowest of the three models, reflecting that it is correct only on the majority class.

Table 3: Split conformal calibration at target coverage $1 - \alpha = 0.9$, calibrated on the validation partition and evaluated on test.

Model	Target cov.	Empirical cov.	Avg. set size
Majority	0.9000	1.0000	3.0000
MultinomialLR	0.9000	0.8922	2.0784
OrdinalLR	0.9000	0.8855	1.7154

Table 3 reports the split conformal calibration at a 90% target coverage, calibrated on the validation partition and evaluated on test. The ordinal model's conformal sets attain empirical coverage close to the 90% target while remaining the most informative: an average set size of 1.72 out of 3 possible classes, versus 2.08 for the multinomial baseline and a trivial 3.0 (every class, always) for the majority baseline. The majority baseline's set size directly reflects that its point probabilities are too flat (identical for every input) to ever confidently exclude a class – it "achieves" coverage only by refusing to narrow the answer at all, the degenerate case the average-set-size metric exists to catch.

7.1 Reading the headline number

The practical takeaway is not "65% accuracy," which undersells what the pipeline actually produces. It is: for a held-out shopper the system never saw during training, the calibrated output is typically a set like $\{\text{fit}\}$ or $\{\text{fit}, \text{large}\}$ with a guarantee that the true outcome falls inside that set at least 90% of the time – a result a single point prediction cannot make and cannot be checked against.

8 Conclusion

We treated clothing-fit prediction as what it structurally is – an ordinal classification problem over three ordered levels – rather than defaulting to an unordered multinomial classifier or collapsing it into a binary signal, and paired the fitted model with split conformal prediction so its output is a calibrated set, not a bare label. Both pieces are classical machine learning: a cumulative link model fit by maximizing an explicit likelihood with `scipy.optimize`, and a closed-form calibration procedure over a held-out partition, implemented from scratch in NumPy. No neural network, embedding layer, or gradient-descent training loop appears anywhere in the pipeline.

The ordinal structure measurably helped: against an unordered multinomial logistic regression trained on identical features, the proportional-odds model nearly halved the mean absolute ordinal error (0.3535 versus 0.6298) and produced visibly tighter conformal sets (1.72 versus 2.08 classes on average) at the same target coverage. Against the naive majority-class floor, it traded a small amount of macro F1 generosity to the majority class for a model that actually uses the shopper's and garment's features, reflected in more than double the majority baseline's macro F1.

8.1 Limitations

Synthetic fixture. All numbers in this report are computed on the synthetic fixture, not the real Kaggle dataset; `scripts/download_data.py -kaggle` provides the path to real data when credentials are available, and the feature pipeline, model, and tests are schema-driven rather than tuned to the fixture's specific distribution.

Proportional-odds assumption. The shared-weight cumulative link model assumes every feature's effect on the cumulative log-odds is constant across thresholds. If, for example, body

type affected the *small* threshold very differently from the *large* threshold, a non-proportional (threshold-varying) extension would fit better; we did not find evidence requiring that extra complexity on this fixture, but a partial-proportional-odds variant is the natural next step on real data.

Conformal set size on a 3-class problem. With only three classes, an "informative" conformal set can only take sizes $\{1, 2, 3\}$; the method scales more informatively to problems with more ordinal levels (e.g. a 5-star or 10-point sizing scale), where a tight conformal set communicates substantially more than it can here.

8.2 What a reader should take away

The point this project makes is not that an ordinal model is marginally more accurate. It is that treating an explicitly ordered outcome as ordered, and reporting a model's own uncertainty honestly instead of suppressing it behind a single confident label, are both small implementation choices with an outsized effect on whether a recommendation can be trusted.

References

[1] Alan Agresti. *Analysis of Ordinal Categorical Data*. Wiley, 2nd edition, 2010.

[2] Anastasios N. Angelopoulos and Stephen Bates. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 16(4):494–591, 2023.

[3] Jing Lei, Max G'Sell, Alessandro Rinaldo, Ryan J. Tibshirani, and Larry Wasserman. Distribution-free predictive inference for regression. *Journal of the American Statistical Association*, 113(523):1094–1111, 2018.

[4] Peter McCullagh. Regression models for ordinal data. *Journal of the Royal Statistical Society: Series B*, 42(2):109–127, 1980.

[5] Rishabh Misra, Mengting Wan, and Julian McAuley. Decomposing fit semantics for product size recommendation in metric spaces. In *Proceedings of the 12th ACM Conference on Recommender Systems (RecSys)*, pages 422–426, 2018.

[6] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. Algorithmic learning in a random world. 2005.